旧版还存在几个必须正面解决的问题。首先，旧版虽然提出了“不确定性感知 + 自适应采样”，但问题定义仍然偏工程优化导向，缺少一个足够锋利、能被审稿人一眼记住的failure mode；其次，旧版的风险函数更像heuristic加权，而不是一个更可解释、更有理论支撑的决策框架；第三，旧版虽然使用了ensemble或dropout来估计不确定性，但没有系统证明这个不确定性本身是可靠的；第四，主动学习还偏“哪里不确定补哪里”，缺少真正面向优化任务本身的utility-aware sampling；第五，实验虽然完整，但还缺少能让baseline系统性失败的困难场景；此外，还需要补上轻量泛化、整体时间成本分析以及动态更新策略的过渡性讨论。这些正是旧版里被明确指出的主要不足。

因此，新版本不能只是在旧版上“加几个模块”，而必须把整篇论文从“一个有效方法”升级为“一个不可替代的问题—方法—系统—实验组合”。换句话说，新架构的首要任务不是让方法看起来更复杂，而是让整篇论文更像是在回答一个明确的问题：为什么现有代理模型方法会在模拟电路优化后期失效，这种失效如何被形式化，如何通过风险感知的模型—仿真协同决策去系统解决，并且为什么这种解决在困难场景下是必要的而不是可选的。旧版已经给出了“问题—方法—系统—实验”四段式贡献组织方式，新版本要做的是把这四段都再压实一层。

新的论文标题层面，建议也从旧版的“Uncertainty-Aware Adaptive Sampling”再往前推进一步，使标题直接体现“可靠优化”与“边界失效”这两个关键词。比如可以考虑：**Reliable Surrogate-Based Analog Circuit Optimization via Failure-Aware Uncertainty Calibration and Utility-Driven Simulation Scheduling**。这个标题相比旧版更强，因为它不再只是强调“有uncertainty和adaptive sampling”，而是显式告诉审稿人：这篇文章处理的是可靠性问题，而且方法上不是简单采样，而是failure-aware和utility-driven的调度。

在新的论文摘要里，故事线也要升级。旧版摘要的重点主要是“通过不确定性和主动学习减少仿真次数”，新版本摘要要先钉死failure mode，再提出方法。也就是说，摘要的前半段不应只说“仿真成本高”，而应明确说：现有代理模型在接近约束边界时容易出现系统性误判，导致后期优化震荡和无效仿真累积，这才是模拟电路代理优化难以真正可靠落地的关键原因。然后再引出新方法：不是直接套一个风险函数，而是引入**failure-aware uncertainty calibration + utility-driven adaptive simulation scheduling**。这里最大的变化是，方法描述要从“预测+不确定性+active learning”三件套，上升为“校准不确定性 + 风险决策 + 任务导向采样 + 闭环在线更新”的统一系统。

具体到新的完整论文架构，建议组织成如下逻辑。

第一章引言，新版本要比旧版更强调“现有方法为什么不够”。旧版已经明确，模拟电路设计是高维、黑盒、仿真昂贵的优化问题，而代理模型方法的真正问题不在平均误差，而在优化后期、特别是接近设计约束边界时的失效。新版引言要把这点进一步展开成三层：第一层，传统代理方法把平均预测精度当作核心目标，但在优化场景下，真正重要的是“是否在高风险点上误判”；第二层，模拟电路的约束天然是硬边界式的，例如phase margin下界、power上界、gain与GBW耦合边界，因此边界附近的一点误差就可能改变可行性判断；第三层，现有方法虽然有BO或uncertainty，但大多没有系统回答“不确定性什么时候可信、何时应该把控制权交还给真实仿真”。引言最后顺势提出本文的三个核心问题：其一，如何形式化后期失效；其二，如何把uncertainty从一个附加信号升级为决策依据；其三，如何在优化任务中动态分配“模型预测”和“真实仿真”的职责。

第二章相关工作，旧版中这一块还没有完全展开。新版本必须把相关工作拆得更细，否则容易被审稿人一句“这不就是BO”打回。建议拆成四类：一类是surrogate-based analog optimization，讨论传统代理模型、贝叶斯优化、进化方法在模拟电路中的应用；一类是uncertainty estimation and calibration，说明ensemble、MC dropout、校准方法在一般ML中的作用以及它们在EDA场景中的不足；一类是active learning for expensive black-box optimization，强调现有方法大多是全局不确定性导向，而非任务效用导向；最后一类是hybrid model-simulation frameworks，说明模型与仿真协同的思想有价值，但在模拟电路可靠优化中仍缺乏一个围绕failure mode构建的完整框架。这样第二章的作用就不再只是“列文献”，而是为第三章的问题定义服务。

第三章问题定义与failure mode分析，这是新版本相较旧版最重要的新增章节。旧版提到了“边界区域误差放大”和“后期失效”，也强调要把failure mode讲透并用实验钉死。新版本必须把这件事从论文的隐含前提变成一个正式章节。这里要做三件事。第一，给出设计任务形式化定义：设计变量 x∈Xx \in \mathcal{X}x∈X，真实性能由 f(x)f(x)f(x) 给出，约束集合定义可行域 F\mathcal{F}F，优化目标是在仿真预算限制下找到满足约束且目标最优的点。第二，定义“后期失效”现象，例如：当候选点接近约束边界时，代理模型对可行性的误判概率显著升高，且这种误判在优化后期会累计并改变搜索轨迹。第三，通过预实验或统计分析证明该现象存在，例如比较不同“边界距离”分桶中的误差分布、可行性误判率、以及纯代理方法后期的震荡行为。这个章节的意义在于告诉审稿人：本文不是拍脑袋加模块，而是在解决一个被明确识别、可测量、且具有普遍性的失效机制。

第四章方法总览，新版本需要把旧版“UAAS”升级成一个更完整的方法框架。旧版的核心思路是：代理模型输出均值和不确定性，结合边界距离形成风险分数，高风险则仿真，低风险则由模型主导。 新版保留这个主线，但把方法拆成四个更独立的模块。第一个模块是**uncertainty estimation and calibration**，不再默认ensemble输出的方差天然可信，而是加入后验校准机制，例如温度缩放式回归校准、quantile calibration，或者至少用保序映射把预测方差和真实误差统计对齐。第二个模块是**failure-aware feasibility risk modeling**，不再只用简单的 R(x)=ασ(x)+β/d(x)R(x)=\alpha \sigma(x)+\beta / d(x)R(x)=ασ(x)+β/d(x) 形式，而是定义“可行性误判风险”或者“边界误导风险”，例如以预测均值、预测方差和到最近约束面的margin为输入，近似计算点被误分类为可行的概率。第三个模块是**utility-driven active sampling**，让采样不再只是选高不确定性点，而是综合考虑该点对最优解搜索的潜在收益。第四个模块是**adaptive simulation scheduling**，根据风险与效用一起决定是“只预测”“先仿真后更新”“批量补点再重训”还是“进入高精度局部搜索”。方法章节中最大的升级，就是把“heuristic risk score”变成“概率化、任务化、决策化”的整体框架，这正是旧版建议中所强调的“从heuristic升级为决策框架”。

第五章数学定义，要比旧版更完整。旧版已经有了基础形式化：设计变量、真实性能函数、代理模型输出均值和不确定性、边界距离、风险函数和决策函数。新版本在此基础上要新增三部分。第一，定义约束可行性概率，例如对第 jjj 个约束 cj(f(x))≤0c_j(f(x)) \le 0cj(f(x))≤0，利用代理模型预测分布近似 P(cj(f^(x))≤0)P(c_j(\hat f(x)) \le 0)P(cj(f^(x))≤0)，并基于此定义联合可行性概率。第二，定义误判风险，例如 Rmis(x)=P(x^ predicted feasible but actually infeasible)\mathcal{R}_{mis}(x) = P(\hat x \text{ predicted feasible but actually infeasible})Rmis(x)=P(x^ predicted feasible but actually infeasible)，哪怕是近似定义，也要把方法从经验加权提升到“最小化误判风险”的角度。第三，定义utility分数，例如结合期望改进、可行性概率、误判风险和模型不确定性，得到新的采样目标 U(x)U(x)U(x)。这样主动学习就从“uncertainty-driven”升级成“utility-driven”，正对应旧版指出的改进方向。

第六章系统实现与GitHub工程架构，新版本也必须扩展。旧版已经给出了一个很好的五层结构：仿真层、数据层、模型层、优化层、评测层，并给出了基础仓库目录。 但为了支撑新版本的failure analysis、uncertainty calibration和utility-driven sampling，GitHub里必须新增一批关键文件和文档。首先，在 `src/evaluation/` 下新增 `failure_analysis.py`、`boundary_behavior.py`、`trajectory_analysis.py`，用来记录和分析后期失效、边界附近误判和优化轨迹震荡。这是新版本相较旧版的第一个核心新增，因为没有这些分析模块，就无法把failure mode“钉死”。其次，在 `src/models/` 或 `src/evaluation/` 下新增 `uncertainty_metrics.py`、`reliability_diagram.py`、`coverage_analysis.py`、`calibration_utils.py`，用于检验不确定性是否真的有意义，包括误差—方差相关性、置信区间覆盖率、校准误差等。这对应旧版中“必须证明不确定性有用”的要求。再者，在 `src/acquisition/` 下除了旧有的 `risk_aware_score.py` 和 `boundary_aware_score.py`，还要新增 `utility_score.py`、`feasibility_risk.py`、`misclassification_risk.py`，让采样器不再只按uncertainty排序，而是按对优化任务真正有价值的点排序。第四，在 `src/optimization/` 下新增 `decision_logger.py` 和 `scheduler.py`，把每一次“为何仿真、为何只预测”的决策记录下来，使系统具有可解释性。第五，在 `src/circuits/` 下新增 `hard_cases.py`，显式构造困难任务，例如极端trade-off、紧约束、多约束冲突等，用于杀手级实验。第六，在 `docs/` 下新增 `failure_mode_analysis.md`、`uncertainty_validation.md`、`hard_case_protocol.md` 和 `wall_clock_analysis.md`。这些新增文档不是附庸，而是把论文里最容易被问倒的四个点——failure mode、uncertainty质量、困难场景、公平时间成本——提前工程化固化下来。

因此，新版GitHub推荐结构不再是旧版那个静态框架，而是升级成下面这种“论文结构与实验逻辑完全同构”的结构：

```
analog-uaas/
├── README.md
├── requirements.txt
├── environment.yml
├── pyproject.toml
├── configs/
│   ├── circuit/
│   │   ├── opamp.yaml
│   │   ├── opamp_hardcase_tightpm.yaml
│   │   └── opamp_hardcase_conflict.yaml
│   ├── model/
│   │   ├── mlp_ensemble.yaml
│   │   ├── mlp_dropout.yaml
│   │   └── calibration.yaml
│   ├── optimizer/
│   │   ├── random_search.yaml
│   │   ├── bo_baseline.yaml
│   │   ├── surrogate_only.yaml
```

```
│   │   ├── uaas_old.yaml
│   │   └── uaas_plus.yaml
│   └── experiment/
│       ├── exp_main.yaml
│       ├── exp_ablation.yaml
│       ├── exp_failure.yaml
│       ├── exp_hardcase.yaml
│       ├── exp_ood.yaml
│       └── exp_wallclock.yaml
├── docs/
│   ├── proposal.md
│   ├── methodology.md
│   ├── simulation_pipeline.md
│   ├── data_schema.md
│   ├── experiment_protocol.md
│   ├── failure_mode_analysis.md
│   ├── uncertainty_validation.md
│   ├── hard_case_protocol.md
│   ├── wall_clock_analysis.md
│   └── paper_figures_plan.md
├── scripts/
│   ├── generate_initial_designs.py
│   ├── run_batch_simulations.py
│   ├── build_dataset.py
│   ├── train_surrogate.py
│   ├── calibrate_uncertainty.py
│   ├── analyze_failure_modes.py
│   ├── run_optimization.py
│   ├── run_hardcase_benchmarks.py
│   ├── evaluate_uncertainty.py
│   ├── evaluate_wallclock.py
│   ├── plot_main_results.py
│   └── reproduce_paper.sh
├── src/
│   ├── circuits/
│   │   ├── base_circuit.py
│   │   ├── opamp_design_space.py
│   │   ├── constraints.py
│   │   └── hard_cases.py
│   ├── simulation/
│   │   ├── simulator_interface.py
│   │   ├── spectre_runner.py
│   │   ├── job_manager.py
│   │   ├── result_parser.py
```

```
│   │   └── failure_checker.py
│   ├── data/
│   │   ├── schema.py
│   │   ├── dataset.py
│   │   ├── normalization.py
│   │   ├── split.py
│   │   ├── boundary_tagging.py
│   │   └── cache.py
│   ├── models/
│   │   ├── mlp.py
│   │   ├── ensemble.py
│   │   ├── mc_dropout.py
│   │   ├── calibration.py
│   │   ├── uncertainty_metrics.py
│   │   └── coverage_analysis.py
│   ├── acquisition/
│   │   ├── expected_improvement.py
│   │   ├── feasibility_risk.py
│   │   ├── misclassification_risk.py
│   │   ├── utility_score.py
│   │   └── boundary_aware_score.py
│   ├── optimization/
│   │   ├── loop.py
│   │   ├── candidate_generator.py
│   │   ├── active_learner.py
│   │   ├── scheduler.py
│   │   ├── trust_controller.py
│   │   └── decision_logger.py
│   ├── evaluation/
│   │   ├── metrics.py
│   │   ├── benchmark.py
│   │   ├── ablation.py
│   │   ├── ood.py
│   │   ├── failure_analysis.py
│   │   ├── boundary_behavior.py
│   │   ├── trajectory_analysis.py
│   │   ├── reliability_diagram.py
│   │   └── visualization.py
│   └── utils/
│       ├── logger.py
│       ├── seed.py
│       └── io.py
├── results/
│   ├── logs/
```

```
│   ├── checkpoints/
│   ├── figures/
│   ├── tables/
│   └── decision_logs/
└── tests/
```

这里面相较旧版新增的关键东西，可以明确概括为五类。第一类是**failure-mode**相关模块，包括 `failure_analysis.py`、`boundary_behavior.py`、`trajectory_analysis.py` 和文档 `failure_mode_analysis.md`。第二类是不确定性可靠性相关模块，包括 `uncertainty_metrics.py`、`coverage_analysis.py`、`reliability_diagram.py`、`uncertainty_validation.md`。第三类是**task-aware / utility-aware** 采样相关模块，包括 `feasibility_risk.py`、`misclassification_risk.py`、`utility_score.py`。第四类是困难场景实验相关模块，包括 `hard_cases.py`、`run_hardcase_benchmarks.py`、`hard_case_protocol.md`。第五类是整体成本与可解释决策相关模块，包括 `scheduler.py`、`decision_logger.py`、`evaluate_wallclock.py`、`wall_clock_analysis.md`。这些新增恰好对应旧版指出的最核心改进方向：failure mode、决策框架、不确定性质量、困难场景、整体时间成本。

第七章实验设置，在新版本中也要扩展为更严谨的实验协议。旧版已经要求固定拓扑、固定工艺、单一对象做深，并强调评价体系必须同时包含效率、质量、稳定性和不确定性质量。新版实验设置要在此基础上明确四类实验组。第一组是主结果实验：比较 random / surrogate-only / BO / uncertainty-only / UAAS+。第二组是failure case实验：专门考察接近phase margin边界、功耗上界或多约束冲突区时，各方法的可行性误判率和后期震荡。第三组是ablation实验：分别去掉校准、不加utility、去掉边界风险、关闭在线更新，以证明各个新增模块确实有贡献。第四组是扩展实验：包括轻量OOD（改变负载或spec）、wall-clock分析，以及动态更新频率分析。这个结构比旧版更强，因为它不只是证明“平均上更好”，而是证明“在最关键、最容易失败的地方更好”。

第八章主结果分析，应围绕“为什么baseline会输、为什么新方法会赢”展开，而不是简单列数字。这里要特别强调三类图。第一类是整体性能图，展示在相同设计目标下，达到可行解所需仿真次数的下降。第二类是优化轨迹图，展示surrogate-only或标准BO在后期围绕边界震荡，而新方法更稳定。第三类是风险—决策图，展示新方法在高误判风险区域会自动增加真实仿真，而不是盲目信模型。第四类是uncertainty质量图，展示高不确定性确实对应高误差，以及经过校准后这种对应关系更强。旧版已经意识到这类可视化的重要性，新版只是把它们升级成论文的主证据。

第九章消融与扩展分析是新版本必须加强的一章。旧版虽然有ablation和OOD意识，但没有完全上升为完整章节。新版本这里要系统回答几个问题：一，若没有校准，仅靠原始ensemble方差是否足够；二，若只做uncertainty-driven active learning，而不做utility-aware是否会浪费仿真预算；三，若只考虑平均预测和边界距离，不建模误判风险，是否仍会出现后期不稳定；四，在线更新频率如何影响性能与时间成本。这一章的作用是把方法从“一个整体黑箱”拆开，证明每个新增模块都不是装饰。

第十章讨论与局限性，新版本也应显式保留。这里可以自然承接旧版提出但尚未完全展开的两点：轻量泛化与动态模型更新策略。可以诚实说明：本文主要聚焦固定拓扑下的可靠优化，尚未覆盖跨拓扑泛化；本文已加入在线更新，但仍主要针对单电路闭环，不涉及更高层级agent决策；本文中

的utility建模和可行性风险仍有进一步理论化空间。这种写法反而会让文章显得成熟，因为它明确为第二篇agent系统留出自然过渡。

因此，新的完整论文架构可以概括为如下顺序：第一章引言；第二章相关工作；第三章问题定义与代理失效分析；第四章方法总览；第五章概率化风险建模与任务导向采样的数学定义；第六章系统实现与GitHub框架；第七章实验设置与协议；第八章主结果；第九章消融、困难场景与扩展分析；第十章讨论与局限；第十一章结论。这个版本相较旧版最本质的变化，不是目录更长，而是多了三个真正决定定会水准的支柱：**failure mode**被正式提出并被实验证明；方法从**heuristic risk score**升级为概率决策与**utility-aware scheduling**；**GitHub**工程从“能跑实验”升级为“能支持科学论证”。